Programmers as code health guardians

Artie Shevchenko

Grupo Sem_ES3_04

Marcelo Díez Domínguez

Daniel López Fernández

Raúl Velasco Vizán

Artie Shevchenko

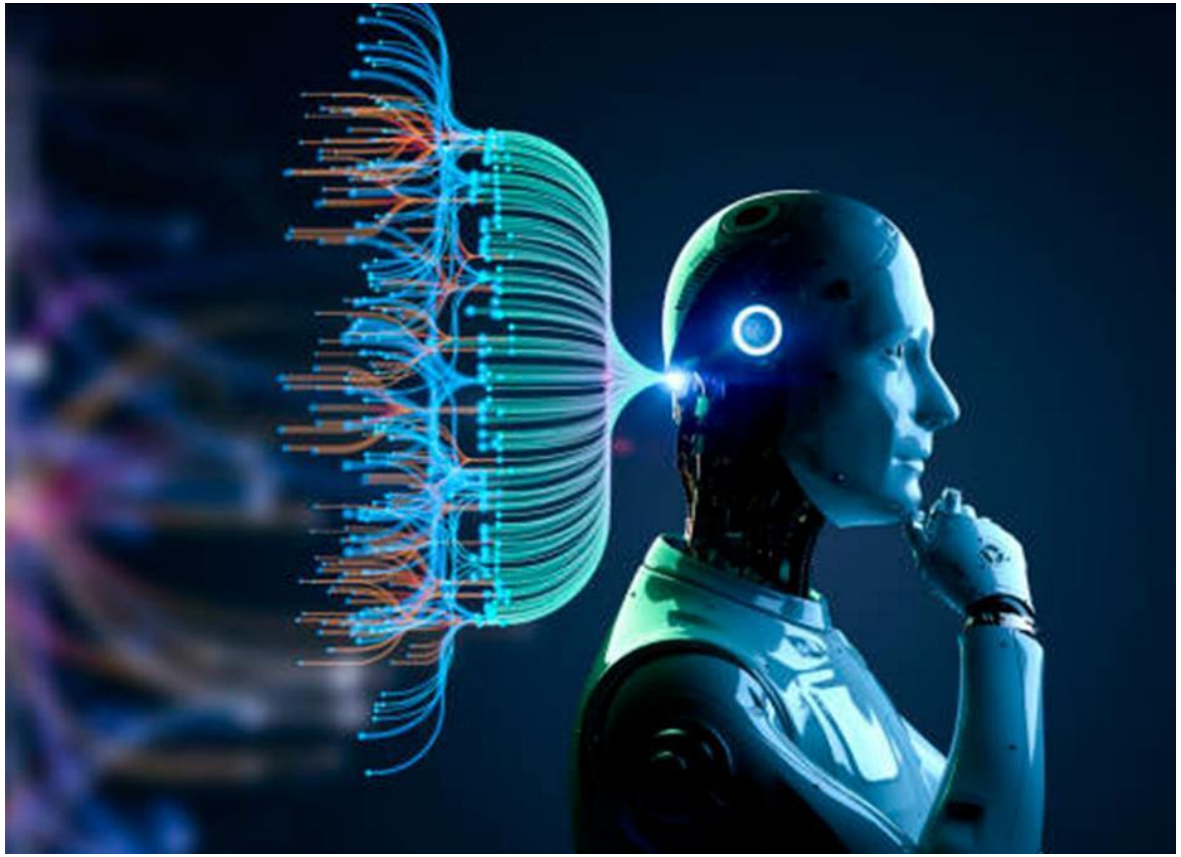
- Antiguo ingeniero de software en Google
- Ex-CEO y fundador de una startup
- Actualmente, ingeniero software en Canva y profesor en la Universidad ITMO
- Autor del libro *Code Health Guardian, the Old New Role of Human Programmers in the AI Era*

¿A qué se refiere con 'code health', y por qué es tan difícil conseguirlo?

'Code health' hace referencia al código simple.

Los problemas complejos requieren ser comprendidos de manera profunda, requieren manejar la complejidad.

¿Qué se necesita para ser exitoso en la era de la IA?



- Colaborar con la IA
- Delegar en la IA
- Ser mejor que la IA en algo (manejar la complejidad)

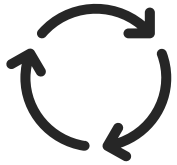
"A veces, la Inteligencia artificial genera soluciones simplistas en vez de simples"

'Term-productivity'

"Encontrar el balance correcto entre la productividad a largo y a corto plazo".

"Creo que la única buena solución aquí es que los ingenieros estén más orientados al producto, que sean más como *product owners*, y que los ingenieros tengan muchas más probabilidades de tener una visión completa del panorama. "

Problemas de la Complejidad



Change Amplification

- Cambios “pequeños”
- Baja velocidad de desarrollo

Cognitive Load

- “*Entender cuesta demasiado*”
- Dos extremos:
  - Megafunción
  - Sobre-descomposición

Unknown unknowns

- “*No sé lo que no sé*”

Causas de la Complejidad

Obscuridad

- Falta de documentación
- Nombres confusos
- Poca trazabilidad

Interfaces complejas

- Demasiados parámetros
- Métodos pocos coherentes

Unfamiliarity / inconsistencia

- Consistencia de estilo
- Consistencia de calidad/diseño

Documentos de diseño

- Ayudan a mejorar la productividad
- La mejor opción son los "one-pager"
- Tienen una plantilla definida
- Son la mejor forma de realizar tradeoff analysis.
- Conocimiento generalizado del proyecto.

	Timely	Familiar process	Second opinion	Inclusive	Everybody on the same page	Clarity	Level of details	Documented	Direct cost, man-hours
"All-hands" meetings	2	3	3	2	2	1	1	2	0
1:1 meetings	3	2	3	0	0	1	2	1	2
In your head prior to coding	3	3	0	0	0	1	2	1	3
When coding	1	3	0	0	0	2	3	1	3
Design doc	**3**	**3**	**3**	**3**	**3**	**2**	**2**	**3**	**1**
Short design doc	**3**	**3**	**3**	**3**	**3**	**2**	**1**	**3**	**2**

Note: Green is good, red is bad.

Table 8.5. Trade-off analysis of different ways of doing trade-off analysis. My take on the exercise.

Porque deberíamos revisar el código

- Existen 4 objetivos principales al realizar revisiones de código:
  - Código de calidad
  - Conocimiento compartido
  - Velocidad de entrega
  - Seguridad psicológica

Interfaces profundas y superficiales

- Módulos profundos y poco profundos o superficiales, término acuñado por John Osterhoutís
- Interfaces sencillas con mucha funcionalidad
- Interfaces complejas con poca funcionalidad

Dudas